

Bases de Datos

Clase 19: Lab — EXPLAIN ANALYZE

Diseño de Índices en la Práctica

Motivación

- En las últimas 3 semanas vimos la teoría: costos I/O, algoritmos, planificación.
- Hoy conectamos con la **práctica**: ¿cómo veo lo que realmente hace PostgreSQL?
- Herramientas: `EXPLAIN` y `EXPLAIN ANALYZE`.
- **Objetivo**: al terminar, podrán diagnosticar una consulta lenta y proponer mejoras concretas con evidencia.

Setup del lab

Creamos una BD limpia y nos conectamos desde Python:

```
import psycopg2, pandas as pd

conn = psycopg2.connect(dbname="lab_explain", host="localhost", user=USER)
conn.autocommit = True

def run(sql):
    with conn.cursor() as cur:
        cur.execute(sql)
        if cur.description is not None:
            cols = [d.name for d in cur.description]
            return pd.DataFrame(cur.fetchall(), columns=cols)

def explain(sql):
    df = run(sql)
    print("\n".join(df.iloc[:, 0].tolist()))
```

El notebook adjunto tiene todo esto listo. Hoy lo corremos en vivo.

Parte 1: EXPLAIN vs. EXPLAIN ANALYZE

Esquema de calentamiento: Capitanes / Botes / Reservas

```
CREATE TABLE Capitanes (cid SERIAL PK, cnombre TEXT, rating INT, edad INT);
CREATE TABLE Botes      (bid SERIAL PK, bnombre TEXT, bcolor TEXT);
CREATE TABLE Reservas   (cid INT FK, bid INT FK, fecha DATE);

-- 1.000 capitanes, 50 botes, 3.000 reservas
INSERT INTO Capitanes ... FROM generate_series(1, 1000);
INSERT INTO Botes      ... FROM generate_series(1, 50);
INSERT INTO Reservas   ... FROM generate_series(1, 3000);
ANALYZE;
```

El esquema clásico de los slides — chico, para ilustrar la salida de `EXPLAIN` .

EXPLAIN vs. EXPLAIN ANALYZE

Comando	¿Ejecuta la consulta?	¿Qué muestra?
EXPLAIN	No	Plan elegido + costos estimados
EXPLAIN ANALYZE	Sí	Plan + costos estimados + tiempos y filas reales
EXPLAIN (ANALYZE, BUFFERS)	Sí	+ páginas leídas del buffer vs. disco

```
EXPLAIN SELECT * FROM Capitanes WHERE rating > 8;  
EXPLAIN ANALYZE SELECT * FROM Capitanes WHERE rating > 8;  
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM Capitanes WHERE rating > 8;
```

EXPLAIN: solo el plan estimado

QUERY PLAN

Seq Scan on capitanes (cost=0.00..21.00 rows=183 width=23)
Filter: (rating > 8)

- Seq Scan : el algoritmo elegido (full scan).
- cost=0.00..21.00 : costo **estimado** (startup..total) en unidades arbitrarias.
- rows=183 : filas estimadas.
- width=23 : ancho promedio (bytes) de cada fila.

→ El plan, pero **sin tocar la BD**.

EXPLAIN ANALYZE: ejecuta y mide

```
Seq Scan on capitanes (cost=0.00..21.00 rows=183 width=23)
    (actual time=0.012..0.387 rows=192 loops=1)
    Filter: (rating > 8)
    Rows Removed by Filter: 808
Planning Time: 0.085 ms
Execution Time: 0.421 ms
```

Campo nuevo	Significado
actual time	Tiempo real (ms): startup..total
rows=192	Filas reales (estimó 183 → error ~5%)
loops=1	Cuántas veces se ejecutó este nodo
Rows Removed by Filter	Filas leídas pero descartadas

EXPLAIN (ANALYZE, BUFFERS)

Conecta con la teoría del **buffer manager**: ¿cuántas páginas vinieron de RAM vs. de disco?

```
Seq Scan on capitanes (actual time=0.012..0.387 rows=192 loops=1)
  Filter: (rating > 8)
  Buffers: shared hit=8
```

- `shared hit=8` : 8 páginas servidas desde el **buffer pool** (RAM).
- `shared read=...` : páginas que tuvieron que leerse de **disco**.

Esta es la métrica que conecta `EXPLAIN` con las fórmulas de I/O del curso.

Cuidado con escrituras

`EXPLAIN ANALYZE` con `INSERT/UPDATE/DELETE` **sí ejecuta** la operación. Para ver el plan sin perder datos:

```
BEGIN;  
EXPLAIN ANALYZE DELETE FROM Reservas WHERE fecha < '2024-02-01';  
ROLLBACK;
```

```
# En psycopg2:  
conn.autocommit = False  
try:  
    explain("EXPLAIN ANALYZE DELETE FROM ...;")  
finally:  
    conn.rollback()  
    conn.autocommit = True
```

Tipos de scan comunes

Tipo	Significado	Cuándo aparece
Seq Scan	Recorrer toda la tabla	Sin índice útil, o tabla pequeña
Index Scan	Usar un índice y luego ir a la tabla	Índice existe y es selectivo
Index Only Scan	Responder desde el índice sin tocar la tabla	Todos los atributos están en el índice
Bitmap Index Scan	Construir bitmap de páginas, luego leerlas	Múltiples condiciones o selectividad media

Parte 2: Leyendo planes de join

Una consulta con join

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT C.cnombre, R.fecha
FROM Capitanes C JOIN Reservas R ON C.cid = R.cid
WHERE C.rating > 8;
```

¿Qué algoritmo elegirá Postgres? ¿Aplicará la selección antes o después del join? ¿Quién va al build side?

El plan resultante

```
Hash Join (cost=27.6..78.1 rows=576) (actual time=0.45..1.92 rows=584 loops=1)
  Hash Cond: (r.cid = c.cid)
    → Seq Scan on reservas r (actual rows=3000)
    → Hash (actual rows=192)
      Buckets: 1024 Batches: 1
      → Seq Scan on capitanes c
        Filter: (rating > 8)
        Rows Removed by Filter: 808
        (actual rows=192)
Planning Time: 0.31 ms Execution Time: 2.05 ms
```

Cómo leer el plan

```
Hash Join
├── Seq Scan on reservas      ← probe side (recorre tupla a tupla)
├── Hash                      ← build side (construye hash table)
│   ├── Seq Scan on capitanes
│   └── Filter: rating > 8 ← selección empujada hacia abajo ✓
```

Observaciones:

1. Postgres eligió **Hash Join** (no NLJ ni SMJ).
2. La selección `rating > 8` se aplicó **antes** del join (push-down).
3. Capitanes (más chica después del filtro) está en el **build side**.
4. La estimación (576) fue muy cercana a lo real (584).

Tipos de join en EXPLAIN

En EXPLAIN	Algoritmo del curso
Nested Loop	NLJ / BNLJ (sin índice) o INLJ (con índice en el inner)
Hash Join	HJ
Merge Join	SMJ

Si ven Nested Loop con un Index Scan adentro, es un **Index Nested Loop Join**.

Preguntas para discutir

1. ¿Postgres eligió Hash Join, Nested Loop o Merge Join? ¿Por qué tiene sentido aquí?
2. ¿La selección `rating > 8` se aplicó antes del join (push-down)?
3. ¿Cuál tabla quedó en el build side y cuál en el probe side? ¿Por qué?
4. ¿La estimación de filas se acercó a la realidad?

Parte 3: Ejercicio en vivo — Efecto de los índices

Setup: empleados (1M filas) y departamentos (200)

```
CREATE TABLE empleados (  
  eid SERIAL PRIMARY KEY,  
  nombre TEXT, depto_id INT, salario INT  
);  
CREATE TABLE departamentos (  
  did SERIAL PRIMARY KEY,  
  nombre TEXT, ciudad TEXT  
);
```

```
INSERT INTO departamentos ...      -- 200 deptos en Santiago/Valparaíso/Concepción  
INSERT INTO empleados ...          -- 1.000.000 empleados  
                                   -- depto_id = (i % 200) + 1  
                                   -- salario = 30000 + random()*70000  
ANALYZE empleados; ANALYZE departamentos;
```

Aquí los datos son grandes — el efecto de los índices se verá de verdad.

La consulta de referencia

```
EXPLAIN ANALYZE
SELECT e.nombre, d.nombre AS depto, e.salario
FROM empleados e JOIN departamentos d ON e.depto_id = d.did
WHERE e.salario > 80000 AND d.ciudad = 'Santiago';
```

La iremos corriendo después de cada cambio de índices y comparando los planes.

Paso 1 — Sin índices adicionales

Solo existen los índices implícitos en las PKs.

Preguntas para la clase:

1. ¿Qué tipo de join eligió PostgreSQL?
2. ¿Dónde aplicó las selecciones (`salario > 80000` , `ciudad = 'Santiago'`)?
3. ¿Usó algún índice?
4. ¿Cuántas filas estimó vs. cuántas produjo realmente?

Paso 2 — Índice en **salario**

```
CREATE INDEX idx_emp_salario ON empleados (salario);  
EXPLAIN ANALYZE  
SELECT e.nombre, d.nombre AS depto, e.salario  
FROM empleados e JOIN departamentos d ON e.depto_id = d.did  
WHERE e.salario > 80000 AND d.ciudad = 'Santiago';
```

Preguntas:

1. ¿Cambió el plan? ¿Usa el nuevo índice? ¿De qué forma (Index Scan, Bitmap)?
2. ¿Mejóro el tiempo?
3. Si **no** lo usó: ¿por qué?

¿Cuánto filtra realmente `salario > 80000`?

```
SELECT
  COUNT(*) FILTER (WHERE salario > 80000) AS sobre_80k,
  COUNT(*) AS total,
  ROUND(100.0 * COUNT(*) FILTER (WHERE salario > 80000)
        / COUNT(*), 1) AS pct_sobre_80k
FROM empleados;
```

sobre_80k	total	pct_sobre_80k
285643	1000000	28.6

→ ~**29%** de las filas pasan. Para Postgres, leer 290k tuplas vía índice (con los saltos al heap) es **más caro** que un seq scan.

Paso 3 — Índice en `depto_id`

```
CREATE INDEX idx_emp_depto ON empleados (depto_id);  
EXPLAIN ANALYZE  
SELECT e.nombre, d.nombre AS depto, e.salario  
FROM empleados e JOIN departamentos d ON e.depto_id = d.did  
WHERE e.salario > 80000 AND d.ciudad = 'Santiago';
```

Pregunta: ¿el optimizador ahora usa **INLJ** con el índice en `depto_id` ? ¿O todavía prefiere Hash Join? ¿Por qué?

Pista: pensar en cuántas veces se invocaría el índice en un INLJ vs. el costo de construir la tabla de hash.

Umbral de selectividad

¿Qué pasa con un filtro **muy** selectivo?

```
-- ~20% de filas → probablemente Seq Scan  
EXPLAIN ANALYZE SELECT * FROM empleados WHERE salario > 60000;  
  
-- ~5% de filas → probablemente Index Scan  
EXPLAIN ANALYZE SELECT * FROM empleados WHERE salario > 95000;
```

Lección: el optimizador solo usa un índice cuando la selectividad lo justifica. Umbral típico: ~5–15% de las filas.

Estadísticas desactualizadas

¿Qué pasa cuando los datos cambian y el optimizador no se entera?

```
-- Insertamos 500k filas con salario alto (sesgo en la distribución)
INSERT INTO empleados (nombre, depto_id, salario)
SELECT 'Nuevo ' || i, (i % 200) + 1, 90000 + (random() * 10000)::int
FROM generate_series(1, 500000) AS i;

-- Deshabilitamos autovacuum para que no se "despierte"
ALTER TABLE empleados SET (autovacuum_enabled = false);
```

Sin ANALYZE: las estimaciones mienten

```
EXPLAIN ANALYZE SELECT * FROM empleados WHERE salario > 95000;
```

Postgres estima usando la distribución **vieja** → cree que hay pocas filas → quizás elige Index Scan. Pero ahora hay muchísimas más → ejecuta lento y la estimación está muy lejos de la realidad.

```
ANALYZE empleados;    -- actualiza estadísticas  
EXPLAIN ANALYZE SELECT * FROM empleados WHERE salario > 95000;
```

→ El plan **cambia**. Las estimaciones se acercan a lo real.

```
ALTER TABLE empleados SET (autovacuum_enabled = true); -- restaurar
```

Forzar un plan diferente (opcional)

PostgreSQL permite **deshabilitar** estrategias para comparar:

```
SET enable_hashjoin = off;  
EXPLAIN ANALYZE SELECT ...;    -- ¿qué elige ahora?  
  
SET enable_hashjoin = on;  
SET enable_mergejoin = off;  
EXPLAIN ANALYZE SELECT ...;    -- ¿y ahora?  
  
RESET ALL;                      -- restaurar
```

Solo para aprendizaje. En producción, dejen que el optimizador decida.

¿Qué significan los "costos" en EXPLAIN?

EXPLAIN reporta costos en **unidades arbitrarias**, no en I/O directamente:

Parámetro	Default	Significado
seq_page_cost	1.0	Costo de leer 1 página secuencialmente
random_page_cost	4.0	Costo de leer 1 página aleatoriamente
cpu_tuple_cost	0.01	Costo de procesar 1 tupla

`cost=0.00..35.50` $\Rightarrow$ 0 de startup, 35.5 totales.

Conexión con la teoría: las fórmulas del curso cuentan páginas; EXPLAIN pondera según tipo de acceso (aleatorio pesa 4× secuencial).

Parte 4: Ejercicio integrador

Tres consultas representativas

Esquema: Capitanes(cid, cnombre, rating, edad), Botes(bid, bnombre, bcolor), Reservas(cid, bid, fecha).

-- Q1: Reservas de un capitán específico

```
SELECT * FROM Reservas WHERE cid = 42;
```

-- Q2: Capitanes con rating alto que reservaron botes rojos

```
SELECT C.cnombre  
FROM Capitanes C, Reservas R, Botes B  
WHERE C.cid = R.cid AND R.bid = B.bid  
      AND C.rating > 9 AND B.bcolor = 'rojo';
```

-- Q3: Número de reservas por bote, ordenado

```
SELECT B.bnombre, COUNT(*)  
FROM Botes B JOIN Reservas R ON B.bid = R.bid  
GROUP BY B.bnombre  
ORDER BY COUNT(*) DESC;
```

Tarea

1. Para cada consulta, **proponer índices** que podrían ayudar.
2. **Justificar** con las fórmulas de costo de las semanas 7–10.
3. **Verificar** con `EXPLAIN ANALYZE` antes y después de crear cada índice.

```
-- Espacio para experimentar:  
-- CREATE INDEX ... ;  
-- EXPLAIN ANALYZE ... ;
```

El objetivo no es "usar más índices", es **defender** cada decisión con evidencia.

Parte 5: Herramientas de producción

`pg_stat_user_indexes` — ¿qué índices nunca se usan?

```
SELECT relname      AS tabla,  
       indexrelname AS indice,  
       idx_scan,  
       idx_tup_read  
FROM pg_stat_user_indexes  
ORDER BY idx_scan ASC, indexrelname;
```

Los índices con `idx_scan = 0` después de un workload representativo son **candidatos a eliminar**: ocupan espacio y ralentizan escrituras sin aportar lecturas.

Tamaño de tablas e índices

```
SELECT relname,  
       pg_size_pretty(pg_relation_size(oid)) AS tamaño  
FROM pg_class  
WHERE relnamespace = 'public'::regnamespace  
      AND relkind IN ('r','i')  
ORDER BY pg_relation_size(oid) DESC;
```

relname	tamaño
empleados	65 MB
idx_emp_salario	32 MB
idx_emp_depto	28 MB
empleados_pkey	21 MB
...	

Los índices **no son gratis**: ocupan disco y RAM en el buffer pool.

Otras herramientas útiles

Herramienta	Para qué sirve
<code>\timing</code> en psql	Medir tiempo total de cualquier query
<code>EXPLAIN (ANALYZE, BUFFERS)</code>	Plan + tiempos + páginas RAM/disco
<code>pg_stat_statements</code>	Ranking de queries más lentas / frecuentes
<code>pg_stat_user_indexes</code>	Índices que nunca se usan
<code>ANALYZE tabla</code>	Actualizar estadísticas → mejores planes
<code>VACUUM</code>	Reclamar espacio de tuplas eliminadas (bloat)

Lecciones aprendidas

Observación	Lección
El optimizador no siempre usa el índice	Si el filtro no es selectivo, Seq Scan puede ser más barato
Agregar un índice no siempre ayuda	Depende de la consulta y la distribución de datos
El orden del join lo elige Postgres	Pero podemos <i>influirlo</i> con índices y selectividad
Las estimaciones pueden errar	<code>ANALYZE</code> mantiene las estadísticas al día
Cada índice tiene un costo de mantenimiento	Eliminar los que no se usan

Conexión con lo que viene: transacciones

El modelo pipelining tiene consecuencias para la **conurrencia**:

- Una consulta larga mantiene páginas y locks **mientras itera tuplas**.
- ¿Qué pasa si **dos consultas** se ejecutan en paralelo y acceden a las **mismas páginas**?
- ¿Y si una está **leyendo** mientras otra **escribe**?

Próxima clase: Transacciones

- ACID y anomalías de concurrencia.
- 2PL (Two-Phase Locking), deadlocks.
- Niveles de aislamiento en PostgreSQL (con sus trampas).

